

Computing Methods for Physics – 13 November 2023

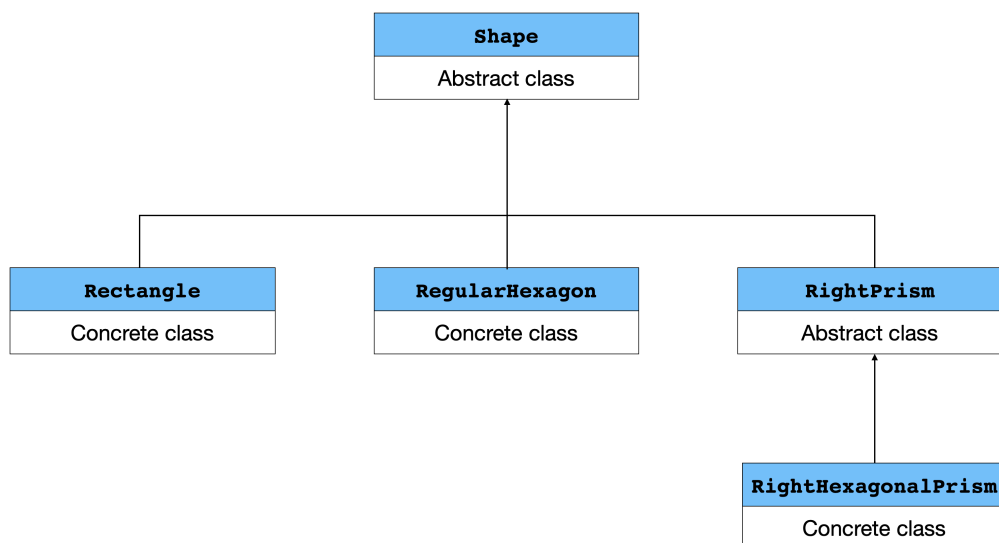
Your exam material (code files, plots, datafiles, etc.) must be submitted via google classroom by 16:00 as a single zip file.

C++ evaluation will be based on: correct syntax, proper return types, proper arguments of functions, data members and class interfaces, setters/getters, unnecessary void functions, correct mathematical expressions, comments throughout the code, separation of class implementations and interfaces.

Python evaluation will be based on: correct syntax, avoiding C-style loops, using Python features in general, comments throughout the notebook/scripts, labels, legends and plot styling and clarity in general.

Part 1 – C++

Implement the following classes and inheritance hierarchy.



- An abstract class **Shape** to represent 2D and 3D geometric shapes. **Shape** must require any concrete class inheriting from it to specify the methods **Perimeter**, **Area** and **Volume**, as well as a **print** method.
- Two concrete classes, **Rectangle** and **RegularHexagon**, that inherit directly from **Shape**. [The **Volume** of 2D shapes simply vanishes.]
- The abstract class **RightPrism** that inherits from **Shape** and **RightHexagonalPrism**, a concrete class that inherits from **RightPrism** and relies on **RegularHexagon** to represent right prisms with an hexagonal base. [For 3D shapes, **Perimeter** and **Area** need to return the total length of the edges and the surface area, respectively.]

Part 2 – Python

CLR is a game played by $n > 2$ players each starting with c coins. The players roll the dice in turns and the outcome of the dice roll determines the fate of their coins. A player that has 3 or more coins rolls 3 dice, a player with 2 coins rolls 2 dice, a player with 1 coin rolls 1 die, and a player without coins cannot roll. Each dice has 6 faces: 1 showing a C, 1 showing an L, 1 showing an R, and 3 showing a dot. For every L/R a player rolls, a coin goes to the player on their left/right; for every C a player rolls, one of their coins is removed from the game; rolling a dot has no consequences. The last player left with coins wins.

- Build a class to represent a die: the tuple of face values will be an attribute, and a fair roll will be a method. [*Use `random` and not `numpy.random` to roll the dice: it is less sophisticated and hence faster.*]
- Build a class to simulate a single CLR game. The class must allow you to simulate games with any $n > 2$ and any $c > 0$, and it must have a `__str__` method to help you debug your code. Make sure a game is played using an instance (or instances, you decide) of the class written to represent a die.
- Run a single CLR game. You should be well below a 100ms run time. [*You can check this in Jupyter with the magic command `%% time`.*]
- Run 10^5 CLR games with $n = 3$ and $c = 20$: gather information on the number of rounds each game took to come to an end. [*Start with fewer games to check your runtimes are not too long.*]
- Plot the density distribution of number of rounds obtained as a histogram.
- Fit the density distribution with a Gaussian function and determine the expectation value of number of rounds to be played, as well as the (1 sigma) uncertainty on this number. [*If your density distribution does not look like a Gaussian, something is wrong with your simulations.*]
- Overlay the density distributions and your fit. Mark the relevant interesting quantities you calculated on the plots.

Recommendations

- Break down a game into many simple actions, and write a method of the class for each one: play a full game, play a single round, check whether there is a winner, roll the dice, process the dice roll, etc.
- Use `numpy` arrays as much as you can.
- Comment your notebook by providing your classes and their methods with help strings (the messages you put at the very top within 3 double quotes).